Enterprise Claude Skills Project Handbook

Prepared for: Akash
Purpose: Plain-English context handbook for the **Enterprise Claude Skills / Enterprise AI Project Management Platform** project
Audience: Founder, PM, operator, engineer, or multi-agent system with no prior knowledge of the project
Based on: Your uploaded project specifications and public official documentation
Snapshot date of source docs: May 25, 2026

---

1) Executive summary

This project is **not** a normal task manager and **not** just a prompt library.

It is an **enterprise AI work orchestration platform** that acts as a **control plane** for Claude Skills, agents, departments, and workflows.

In simple terms, the platform should let a company:

- organize work into workspaces, projects, tasks, approvals, dashboards, and automations
- create or upload **skills** and bundle them into reusable business capabilities
- assign those skills to **agents**
- enable or disable capabilities by **department**, **industry**, **workspace**, **customer**, **user role**, or **project**
- meter usage through **skill credits** and subscriptions
- enforce **approval rules**, **audit logs**, **security checks**, and **data boundaries**
- integrate with external tools such as Asana, monday.com, GitHub, Slack, and MCP-compatible systems

The product vision is that a business does not buy "prompts." It buys a **governed AI operating system** for its teams.

---

2) What the project is trying to solve

Most organizations have three problems with AI work today:

1. **AI usage is fragmented.** Teams use random prompts, random tools, and random files.
2. **There is no governance.** Nobody knows which AI workflow is approved, risky, outdated, or expensive.
3. **Capabilities are not packaged as products.** A workflow for marketing, HR, legal, or engineering is usually recreated every time.

This project solves that by turning AI capabilities into **licensed, governed modules**.

Instead of saying:

> "Use this prompt for campaign planning."

The platform should let the company say:

> "Enable the Marketing suite, enable the Retail overlay, allow only approved campaign skills, require legal approval for external claims, charge credits based on complexity, log every run, and keep all outputs tied to the right workspace."

That is why the uploaded catalog describes the system as a **Project Manager AI Control Plane**.

---

3) The simplest mental model

Think of the platform as a combination of five things:

1. **A project management app**
   for workspaces, projects, tasks, approvals, dashboards, comments, documents, and automations.
2. **An agent platform**
   for creating AI agents with defined permissions, tools, memories, and responsibilities.
3. **A skill registry**
   for storing and governing reusable Claude Skills and internal skill packages.
4. **A licensing and billing system**
   for department suites, industry overlays, subscriptions, credits, and agent seats.
5. **A governance layer**
   for approvals, audit logs, risk tiers, security scans, sandboxing, policy enforcement, and controlled execution.

If someone asks what this product is in one sentence, use this:

> **A multi-tenant enterprise AI work orchestration platform that packages Claude Skills into licensed department and industry solutions, routes work to agents, meters usage, and enforces governance.**

---

4) What Claude Skills are, in practical terms

Claude Skills are reusable capability folders that package:

- instructions
- metadata
- optional scripts
- reference files
- templates
- supporting resources

The official Anthropic model is filesystem-based. A skill is essentially a folder with a `SKILL.md` file and optional code/resources. Metadata is lightweight and always known; the main instructions are loaded when the skill is triggered; additional scripts and resources are accessed only when needed.

Why that matters for this project:

- skills become **modular business capabilities**
- you can reuse them across many tasks and users
- you can attach policy, governance, costs, and approvals to them
- you can compose multiple skills into one agent workflow

This is why your project treats skills as **product units**, not as isolated prompts.

---

5) The main business model

The uploaded catalog is very clear: the platform should sell **capabilities**, not generic seats only.

The commercial layers

A. Base platform The customer buys the core **Project Manager AI Control Plane**.

That base platform includes the system needed to:

- create organizations and workspaces
- manage users and roles
- manage tasks, projects, workflows, and files
- register agents and skills
- route work
- log runs
- apply approvals and policies
- measure usage and billing

B. Department suites A department suite is a packaged operating system for a specific business function.

Examples:

- Marketing and Growth
- Sales and Revenue
- Customer Service and Customer Success
- HR, People, and Talent
- IT and Digital Workplace
- Engineering and Product Development
- Product Management
- Design and Creative Studio
- Finance and Accounting
- Operations and PMO
- Data, Analytics, and Business Intelligence
- Legal, Compliance, and Risk
- Procurement and Supply Chain
- Security and GRC
- Executive Strategy and Chief of Staff
- Internal Communications and Corporate Affairs
- Learning, Enablement, and Training
- Facilities, Real Estate, and Workplace Operations
- Research, Innovation, and R&D
- Administration, Executive Assistance, and Office Ops
- Product Marketing and Go-to-Market
- Quality, Regulatory, and Business Assurance

Most department suites are modeled as **48 skill SKUs**. Marketing is larger and is modeled as **72 skill SKUs**.

C. Industry overlays An industry overlay adds sector-specific language, templates, workflows, policies, and dashboards on top of department suites.

Examples:

- Commercial Real Estate and Property Management
- Healthcare and Life Sciences
- Financial Services, Banking, and Insurance
- Technology / SaaS / Software
- Manufacturing and Industrial
- Construction / Architecture / Engineering
- Retail and E-commerce
- Education
- Media and Entertainment
- Professional Services and Consulting
- Government and Public Sector
- Nonprofit / Foundation / Association
- Energy / Utilities / Infrastructure
- Transportation / Logistics / Warehousing

- Hospitality / Restaurants / Travel
- Telecommunications and Connectivity
- Consumer Packaged Goods and Food/Beverage
- Automotive and Mobility
- Pharma / Biotech / Medical Devices
- Agriculture / Food Production / Agribusiness

The uploaded catalog models these as **20 industry overlays**, each with **24 skills**.

D. Skill credits A skill run is billable.

The proposed meter is a **credit model**. The uploaded catalog suggests rough bands such as:

- 1 credit for simple drafting or classification
- 3 to 5 credits for structured work
- 8 to 15 credits for deeper research or analysis
- 25+ credits for regulated or high-risk workflows

E. Agent seats The platform can also charge for named or pooled AI agents.

F. Enterprise / regulated controls Higher tiers add:

- approvals
- audit trails
- compliance evidence
- stronger data boundaries
- security review
- regulated workflow controls

---

6) Who the platform is for

This is an **enterprise B2B platform**.

It is for organizations that want AI inside their operating model, but with control.

Typical buyers include:

- operations leaders
- PMO / chief of staff offices
- CIO / CTO / CISO offices
- marketing and sales leadership
- HR and people operations
- finance, procurement, and legal teams
- compliance, quality, and regulatory teams

- agencies or implementation partners

Typical internal users include:

- admins
- team leads
- project managers
- specialists within each department
- reviewers and approvers
- agent designers / automation builders

---

7) What a "department suite" actually contains

A department suite is not just a label. It should include:

- approved skills
- approved agents
- default task templates
- forms and intake workflows
- dashboards and saved views
- automation rules
- permissions and data boundaries
- approval policies
- billing and cost tracking
- default reports and artifacts
- possible industry overlay compatibility

Example: Marketing and Growth suite

The catalog breaks marketing into many modules, including:

- strategy and planning
- brand and messaging
- graphic design and visual production
- video design and production
- content marketing and editorial
- SEO / AEO / AI search
- paid media and performance marketing
- lifecycle and email marketing
- social and community
- events and field marketing
- PR / analyst / external communications
- marketing operations and analytics

That is why Marketing is larger than other suites.

Example: Engineering suite

The Engineering and Product Development suite includes modules such as:

- planning and specs
- frontend development
- backend and APIs
- database and migrations
- testing and QA automation
- code review and quality
- security and performance
- CI/CD and release

Example: Legal suite

The Legal, Compliance, and Risk suite includes modules such as:

- legal intake
- contracts
- privacy
- compliance programs
- risk management
- litigation and disputes
- policy governance
- AI and agent governance

This shows the platform is meant to map directly onto real business teams.

---

8) What an "industry overlay" actually contains

An industry overlay is a specialization layer.

It should add:

- industry vocabulary
- common process templates
- department recommendations
- domain dashboards
- compliance checks
- domain-specific skill routing
- role mapping
- data-field conventions
- domain metrics
- example workflows

Example: a telecom overlay should add things like network rollouts, field service, outage workflows, regulatory filing calendars, and enterprise sales motions.

Example: a hospitality overlay should add banquets, seasonal promotions, occupancy and labor reporting, menu costing, and supplier workflows.

This means the product can sell one underlying platform across many verticals without rebuilding the whole system.

---

9) Core product requirements

The uploaded documents imply a very large enterprise application. The main requirements can be grouped like this.

A. Enterprise workspace and identity

The platform must support:

- organizations / tenants
- workspaces
- departments / silos
- projects
- tasks and subtasks
- roles and permissions
- user invitations
- approvals
- audit logs
- enterprise auth readiness
- isolation by tenant and workspace

B. Work management primitives

The platform must support:

- tasks
- subtasks
- dependencies
- assignees
- labels
- comments
- attachments
- checklists
- templates
- custom fields
- saved views
- dashboards
- automations
- states / statuses
- portfolio-like groupings

C. Agent creation

The platform should support three creation paths:

1. **Manual skill upload**
   upload or link a reviewed skill bundle.
2. **Skill-composed agent creation**
   pick approved skills and define tools, memory scope, autonomy, and review policy.
3. **SOP-based agent creation**
   upload SOPs, checklists, or process docs and package them into reusable internal skills/agents.

D. Brand voice and AI search

The platform should include:

- tenant-specific brand voice knowledge
- semantic + lexical search across tasks, files, docs, SOPs, comments, and knowledge base material
- document retrieval for agent execution
- knowledge scoping by workspace or tenant

E. Licensing and entitlements

The platform must be able to:

- enable or disable suites
- enable or disable overlays
- meter skill usage
- meter agent usage
- apply subscription tier limits
- apply workspace-level entitlements
- support activation and deactivation controls

F. Audit and governance

The platform must capture:

- who ran what
- which skill was used
- which agent used it
- what files or sources were used
- what approvals were needed
- what outputs were produced
- what cost was charged
- what risk tier applied

---

10) The main system modules

Here is the cleanest module breakdown for implementation.

10.1 Core platform and tenancy

- organizations
- workspaces
- users
- roles
- permissions
- memberships
- billing hooks
- settings

10.2 Project and task management

- projects
- tasks
- subtasks
- labels
- statuses
- comments
- attachments
- forms
- dashboards
- automation rules

10.3 Skill registry

- skill sources
- skill packages
- skill versions
- status lifecycle
- approvals
- provenance
- commit pinning
- hashes
- sandbox policy
- trust score

10.4 Department suite engine

- department definitions
- included skills
- included agents
- billing model

- activation rules
- dashboards and templates

10.5 Industry overlay engine

- overlay definitions
- compatible departments
- overlay-specific skills
- terminology packs
- compliance rules
- field mappings

10.6 Agent system

- agent profiles
- skill attachments
- autonomy level
- tool access
- memory scope
- review agent
- run policies
- handoff rules

10.7 Skill run orchestration

- queueing
- routing
- execution records
- retries
- approvals
- metering
- artifact storage
- traceability

10.8 Approval and policy engine

- risk tiers
- approval gates
- reviewers
- escalation paths
- policy bundles
- blocked actions

10.9 Audit and observability

- immutable audit logs
- LLM run logs

- error logs
- cost records
- system metrics
- traces and alerts

10.10 Integration layer

- Asana
- monday.com
- GitHub
- Trello
- Slack
- MCP servers
- webhooks
- OAuth / token vaulting

10.11 Admin console

- tenant setup
- suite activation
- overlay activation
- skill governance
- approval review
- connector health
- usage analytics
- billing controls

10.12 End-user app

- dashboard
- work views
- task actions
- agent collaboration
- approvals
- reports
- search
- AI assistance inside work

---

11) Why the project needs both admin and end-user apps

The uploaded specs strongly imply two different operating surfaces.

Admin surface

This is where the organization controls the system:

- user and tenant setup
- plan entitlements
- suite activation
- skill governance
- source and integration setup
- billing and usage analytics
- approvals and policy administration
- security review

End-user surface

This is where everyday work happens:

- projects and tasks
- agent-assisted execution
- report generation
- approval routing
- knowledge lookup
- collaboration
- dashboards and work status

This split is important because the person who approves a skill package is often not the same person who uses it in daily work.

---

12) What "silos" mean in this system

The word **silo** in your project should be interpreted as a **department capability boundary**.

A silo is not necessarily a bad thing here. It is a packaging and governance unit.

Each silo should have its own:

- templates
- fields
- dashboards
- roles
- default agents
- skill catalog
- approval rules
- data visibility policies
- budget / cost center logic

For example:

- HR should not automatically have access to Engineering deployment skills.
- Legal should not automatically have access to Marketing external-send skills.

- Finance workflows may require stronger review than content-drafting workflows.

So silos help the platform reflect the real structure of an enterprise.

---

13) Recommended architecture

The product should be built as a **multi-tenant enterprise SaaS platform**.

Recommended application stack

A practical stack for this product would be:

- **Frontend:** Next.js + TypeScript
- **Backend:** NestJS or FastAPI (both are plausible; the uploaded PM spec leans toward a strong API/control-plane design)
- **Database:** PostgreSQL
- **Cache / queues:** Redis + BullMQ or equivalent
- **Search:** PostgreSQL FTS first, then OpenSearch if scale demands it
- **Vector / semantic search:** pgvector initially
- **Object storage:** S3-compatible storage for files and artifacts
- **Observability:** OpenTelemetry + Sentry + Prometheus/Alertmanager

Why PostgreSQL matters here

The platform is heavily relational.

It needs to model:

- tenants
- users
- workspaces
- tasks
- approvals
- entitlements
- skills
- skill runs
- agents
- audit logs

PostgreSQL is well suited for this.

Why object storage matters

The platform will store:

- uploaded files
- skill bundles

- SOP packages
- generated artifacts
- exports
- report outputs

Why pgvector / hybrid search matters

This product needs AI search over:

- docs
- tasks
- comments
- SOPs
- knowledge chunks
- brand voice material
- prior outputs

A hybrid lexical + semantic design is the best fit.

---

14) MCP in this project

MCP matters because your platform is not just another app. It is supposed to act as an **AI control plane**.

Model Context Protocol allows systems to expose:

- **resources** – context objects
- **prompts** – reusable structured workflows
- **tools** – callable actions

For this platform, that means:

Resources

Could include:

- project state
- task state
- SOP bundles
- brand guides
- department policies
- entitlements
- risk policies
- integration metadata

Prompts

Could include reusable workflow templates such as:

- create PRD
- launch campaign
- review contract
- triage incident
- prepare executive update
- create onboarding plan

Tools

Could include actions such as:

- create task
- update task
- assign task
- add comment
- approve run
- reject run
- search knowledge base
- sync external board
- trigger agent

This makes MCP a natural fit for the platform's integration and orchestration layer.

---

15) External integrations the project should support

The uploaded and public references point to these key ecosystems.

Asana

Useful for:

- reading and updating work graph objects
- syncing project/task structures
- MCP-enabled interactions

monday.com

Useful for:

- board and item sync
- AI-assisted workflow operations
- MCP-based tooling

GitHub

Useful for:

- issues
- pull requests
- code review workflows
- engineering operations
- MCP-based repository actions

Trello

Useful for lighter-weight board integrations and card workflows.

Slack

Useful for:

- notifications
- approvals
- conversational agent workflows
- work summaries and routing

Google Workspace and docs ecosystems

Useful for:

- document generation
- spreadsheets
- calendars
- drive-like storage
- presentation workflows

These integrations should be implemented with a strong permission model and auditable actions.

---

16) Skill lifecycle and governance requirements

This project must treat skills as governed runtime assets.

A strong lifecycle is:

- **draft**
- **scanned**
- **reviewed**
- **enabled**
- **disabled**
- **deprecated**
- **quarantined**
- **archived**

For every skill, the system should store:

- source or origin
- version
- commit pin / hash if imported
- trust or review state
- associated department(s)
- associated overlay(s)
- risk tier
- allowed tools
- allowed environments
- required approvals
- artifact outputs

This is critical because public community skills should **not** be trusted automatically.

The uploaded materials and official enterprise guidance both point to a model where public skills are seed material, then rewritten or reviewed into an internal registry.

---

17) Risk tiers and approval model

The uploaded project materials consistently imply a tiered execution model.

A practical model is:

Tier 0 — Read only

- summarize
- classify
- search
- report on existing data

Tier 1 — Workspace write

- create task
- update task
- add comments
- update fields

Tier 2 — External draft

- draft an email
- draft external copy
- draft proposal materials
- prepare export artifacts for review

Tier 3 — Integration write

- modify external systems
- push updates to boards
- create tickets in connected tools

Tier 4 — Controlled execution

- run scripts
- operate on files
- trigger complex flows
- execute guarded system actions

Tier 5 — Regulated / high risk

- external send
- legal or financial commitments
- billing or auth changes
- destructive actions
- sensitive data export
- code execution in production-adjacent contexts

High-risk actions should require human approval.

---

18) Logging and observability requirements

The system should keep three major log families.

A. Audit logs

Track:

- user actions
- admin changes
- skill activation
- suite activation
- approvals
- policy changes
- workspace changes
- exports
- integration changes

B. LLM run logs

Track:

- model provider

- model name
- prompt hash
- tools used
- input artifact hashes
- output artifact hashes
- token counts
- cost
- latency
- run status
- trace ID

C. Error logs

Track:

- integration failures
- retry exhaustion
- model failures
- parser errors
- policy denials
- webhook failures
- worker failures

The logs should be correlated so a human can answer questions like:

- Which skill ran?
- Why did it run?
- Which policy applied?
- Who approved it?
- What output was created?
- What did it cost?
- Why did it fail?

---

19) Data model: the most important core objects

At minimum, the data model should include:

Platform and tenancy

- organizations
- workspaces
- users
- roles
- permissions
- memberships

Work system

- projects
- tasks
- subtasks
- comments
- labels
- attachments
- forms
- dashboards

AI control plane

- skill_sources
- skills
- skill_packages
- department_suites
- industry_overlays
- license_entitlements
- agent_profiles
- skill_runs
- approval_gates
- audit_logs

Knowledge and AI

- brand_voice_documents
- knowledge_chunks
- RAG indexes
- AI prompts
- artifacts

Commercial and policy layer

- subscriptions
- credit_pools
- usage_ledgers
- approval_policies
- risk_policies
- integration_connections

---

20) How custom agents should work

The platform should allow companies to create their own agents.

Each agent should have:

- a name
- a description
- assigned department/silo
- assigned skills
- allowed tools
- memory scope
- data access scope
- approval rules
- review or fallback agent
- autonomy level

Good example agents

- Brand Guardian
- Proposal Writer
- Sales Ops Analyst
- Contract Intake Reviewer
- Release Readiness Agent
- Incident Triage Agent
- Executive Update Agent
- PMO Status Agent
- SEO Audit Agent
- Customer Support QA Agent

Good agent creation modes

1. upload custom skill bundle
2. compose from approved skills
3. generate from SOP and forms

This makes the product useful for both technical and non-technical organizations.

---

21) Why this project is hard

This project is hard because it is really **four products combined into one**:

1. **Project management product**
2. **AI agent platform**
3. **Enterprise governance platform**
4. **Commercial packaging and billing platform**

Many teams can build one of those. Fewer can build all of them together.

The challenge is not just making agents run. It is making them:

- safe
- billable
- reviewable
- explainable
- tenant-aware
- department-aware
- integration-aware
- easy to operate

---

22) What absolutely must be true in version 1

Version 1 does **not** need every feature under the sun, but it does need a complete core loop.

Version 1 should prove these things

1. A customer can create a tenant and workspace.
2. The customer can activate a department suite.
3. The customer can activate an industry overlay.
4. The workspace can use approved skills and agents.
5. Tasks can be created and routed to agents.
6. Runs are logged and metered.
7. High-risk actions require approvals.
8. Skills can be enabled, disabled, and governed.
9. Admins can see audit trails and usage.
10. The system can integrate with at least a few real external tools.

If those ten things work, the platform has a real enterprise core.

---

23) Suggested v1 scope

A smart v1 would likely include:

Core platform

- multi-tenant org/workspace model
- user auth and roles
- projects/tasks/comments/files
- admin UI and end-user UI

Core AI plane

- skill registry
- agent profiles
- skill run tracking
- approval engine
- audit logs
- billing / credit hooks

Commercial packaging

- a few department suites first
- a few industry overlays first
- entitlement logic

Search and knowledge

- tenant docs
- brand voice
- hybrid search

Initial integrations

- GitHub
- Slack
- Asana or monday.com

Initial department suites

A realistic first launch subset might be:

- Marketing and Growth
- Engineering and Product Development
- Product Management
- Operations and PMO
- Legal / Compliance / Risk

Initial overlays

A realistic first launch subset might be:

- Technology / SaaS
- Professional Services
- Healthcare / Life Sciences
- Government / Public Sector

That is still large, but much more realistic than trying to launch every department and vertical at once.

---

24) Suggested build order

A good build order is:

Phase A — Foundation

- repo
- auth
- tenancy
- task model
- admin shell
- end-user shell
- audit log skeleton

Phase B — AI control plane

- skill registry
- skill lifecycle
- agent profiles
- skill run ledger
- approval gates
- risk tiers

Phase C — Packaging and billing

- department suites
- industry overlays
- entitlements
- credits
- usage summaries

Phase D — Knowledge and search

- docs/files
- brand voice
- hybrid search
- retrieval APIs

Phase E — Integrations

- GitHub
- Slack
- Asana or monday.com
- webhook engine

Phase F — UX hardening

- dashboards

- approval queue
- run history
- cost views
- admin policy screens

Phase G — Scale and governance hardening

- stronger security review
- skill scanning
- sandbox policy
- enterprise auth hardening
- export controls

---

25) What a good issue or multi-agent prompt should emphasize

Since you are sending this to an LLM / multi-agent system, every implementation issue should explicitly say:

- **implement real code, not markdown only**
- create actual models, APIs, migrations, services, tests, and UI
- do not stop at planning notes
- preserve tenant isolation
- preserve auditability
- preserve approval flow
- preserve skill lifecycle and provenance
- keep third-party skills behind review gates

That is important because otherwise many agents will stop at "architecture docs" instead of building working foundations.

---

26) Main risks and things to watch out for

Risk 1 — It becomes just another PM app

If the AI control plane is weak, the product becomes a task manager with AI flavor.

Risk 2 — It becomes just a skill marketplace

If the work-management side is weak, the product becomes a loose catalog without business process value.

Risk 3 — Governance is too weak

If approvals, audit logs, and skill vetting are weak, enterprises will not trust it.

Risk 4 — Scope explosion

Because there are many suites and overlays, the team can try to build too much too early.

Risk 5 — Public skill trust

Community skills are useful references, but they should not be auto-installed into production.

Risk 6 — Billing complexity

Licensing by suite + overlay + credits + seats is powerful, but it complicates entitlements and reporting.

---

27) What success looks like

A successful version of this platform would let a company say:

- "We enabled the Marketing suite in three workspaces."
- "We added the Healthcare overlay for one enterprise client."
- "Our Brand Guardian and Campaign Planner agents can draft, but not publish."
- "Legal approval is required for external claims."
- "GitHub and Asana sync into our work system."
- "Every skill run is logged, costed, and reviewable."
- "Our teams reuse vetted workflows instead of rewriting prompts."

That is the real business value.

---

28) Plain-English glossary

Agent

A configured AI worker with specific permissions, tools, and skills.

Skill

A reusable capability package for Claude, usually containing instructions and optional code/resources.

Skill SKU

A productized internal business skill offered as a specific capability.

Department suite

A packaged set of skills, agents, templates, and rules for one business function.

Industry overlay

An add-on that makes department suites industry-specific.

Control plane

The system that governs activation, routing, metering, logging, and policy.

Credit

A billable run unit.

Entitlement

A rule that says what a tenant or workspace is allowed to use.

Risk tier

A classification of how sensitive or dangerous a run is.

Approval gate

A required human decision before a sensitive action can complete.

MCP

Model Context Protocol, a standard for exposing resources, prompts, and tools to AI systems.

Brand voice RAG

A retrieval-backed system that gives the AI company-approved language and writing standards.

---

29) Recommended summary for sharing internally

If you need to explain the project to someone quickly, use this:

> We are building a multi-tenant enterprise AI work orchestration platform. It combines project management, agent execution, Claude Skills governance, billing, and approvals into one system. Companies will buy a base control plane, then activate department suites and industry overlays. Skills and agents are licensed, governed, auditable, and meterable. The system integrates with external work tools and supports custom agents built from approved skills or SOPs.

---

30) Final recommendation

Do **not** treat this as a normal SaaS CRUD app. Do **not** treat it as a loose AI prompt catalog. Do **not** treat it as only an MCP wrapper.

Treat it as an **enterprise AI operating system for work**.

That mindset will keep architecture, permissions, approvals, billing, and agent design aligned.

---

31) Reference notes from official/public sources

Official and public references that matter most for understanding the project:

- Anthropic Agent Skills overview
- Anthropic Skills for enterprise guidance
- Anthropic engineering article on Agent Skills
- Anthropic public skills repository
- Model Context Protocol specification
- Official Asana MCP documentation
- Official monday.com MCP documentation
- Official GitHub MCP server repository

These should be treated as the primary public references for what Claude Skills and MCP can do today.

---

32) Important distinction: this project vs your finance research project

This handbook is about the **Enterprise Claude Skills / AI Project Management Platform** project.

It is **different** from your other project, the **Enterprise Intelligence and Investment Research Platform**.

That finance/intelligence project is about:

- public records
- investment research
- evidence-backed reports
- market data
- legal/regulatory exposure
- intelligence graph

This current project is about:

- work orchestration
- skills packaging
- agent routing
- enterprise governance
- licensing and billing
- department and industry operating models

Do not mix the two when sending instructions to LLMs or developers.

---

33) References

Uploaded project documents used as core context

- Enterprise Claude Skills Catalog - Expanded Department and Industry Suites (your uploaded specification)
- Enterprise AI Project Management Platform Specification (your uploaded operating-model summary)

Official / public reference links

- Anthropic Agent Skills Overview: https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview
- Anthropic Skills for Enterprise: https://platform.claude.com/docs/en/agents-and-tools/agent-skills/enterprise
- Anthropic engineering article: https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills
- Anthropic public skills repository: https://github.com/anthropics/skills
- MCP specification (tools): https://modelcontextprotocol.io/specification/2025-06-18/server/tools
- Asana MCP server docs: https://developers.asana.com/docs/mcp-server
- monday.com MCP servers docs: https://developer.monday.com/apps/docs/mcp-servers
- GitHub official MCP server: https://github.com/github/github-mcp-server

---

34) Closing note

You said you had **no prior knowledge** of the project.

If you remember only four things, remember these:

1. This is an **enterprise AI control plane**, not just a task app.

2. The core commercial objects are **department suites**, **industry overlays**, **agents**, and **skill credits**.

3. The core technical requirements are **governance**, **auditability**, **approvals**, **routing**, and **entitlements**.

4. The build should start small, but the architecture must be designed to support a large enterprise catalog over time.